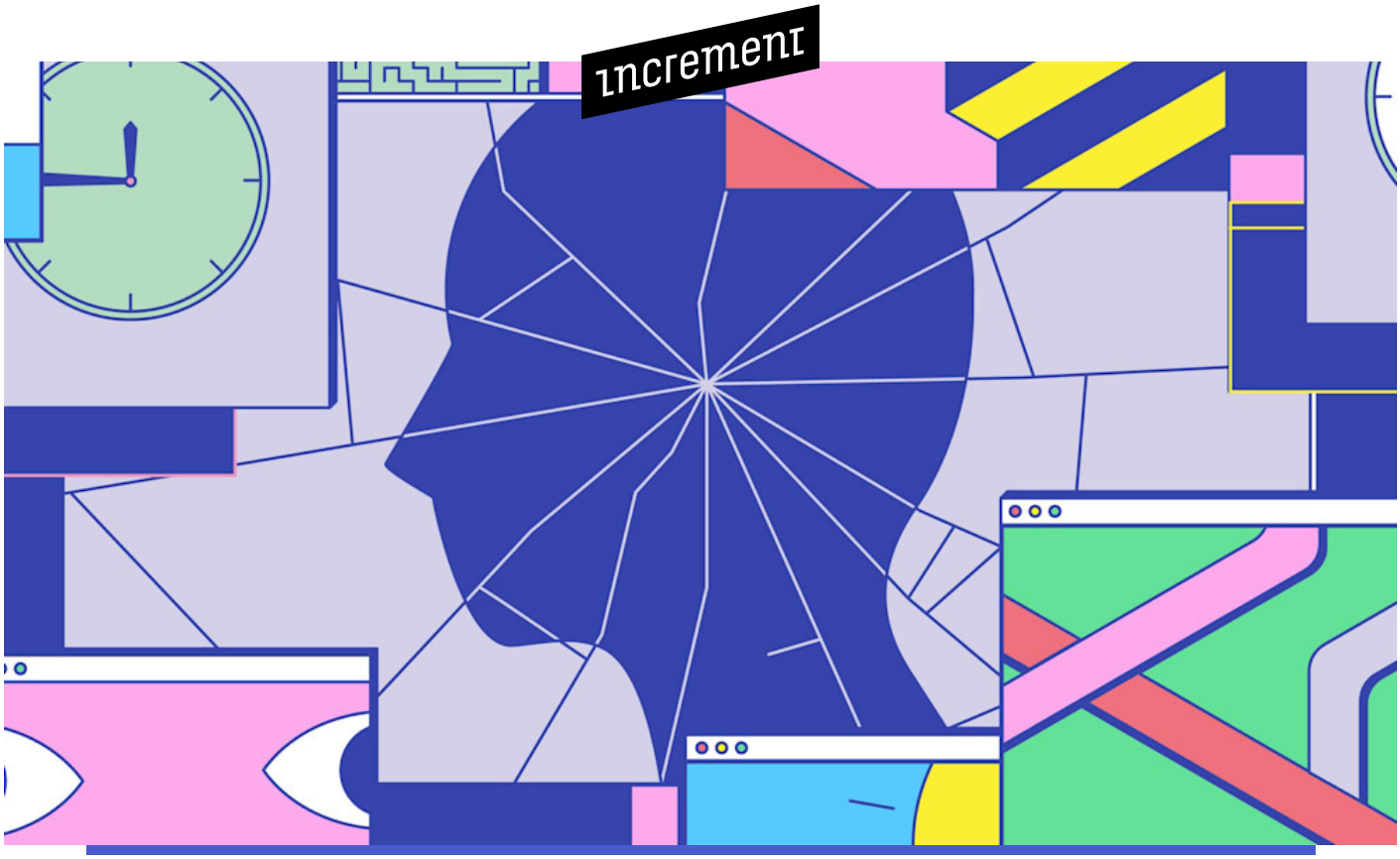


HENRY ZHU

# (Open) source of anxiety

A maintainer of Babel navigates the boundaries of open source and self.

PART OF

ISSUE 9
MAY 2019
Open Source

Since I left my job at Adobe back in March 2018 to do open source "full time" (mostly on Babel, a JavaScript compiler for the existing standard and the future of the language), I've become aware of an off-and-on feeling of anxiety.

I was initially drawn in by the culture of open source. It offered an opportunity to belong to something bigger than myself (that is, it has a community), to live values that combat be-all and end-all consumerism and individualism (that is, it prizes temperance), and to embody a spirit of service parallel to my faith (that is, it's elementally charitable). As a niche of the tech sector, it's surprising how it can be so different from—although totally impacted by—venture capital and startup culture. When I talk about open source, it's always a revealing conversation, whether I'm speaking to someone in or outside of tech: "And you do this for free?"

In open source it looks, at first glance, like just the *code* is open, but its public nature also means that I feel more exposed. In a culture oriented toward (even obsessed with) work, what and how much you produce says everything about you. Maintainers are figures of authority and respect as stewards of the infrastructure of the internet. But now that I am one, I find I'm always in putting-out-fires mode, more reactive than proactive. I often feel a lack of control.

In 2018, for example, many users had enabled JavaScript's future proposed syntax with Babel. The Babel team knew that they were trying out what's called "Stage 0" syntax (stages range from 0 to 4) and that, as a result, they were relying on inherently unstable code in production. It wasn't that the code itself was wrong, but that—because it was not yet a standard—it would inevitably change. People would have to migrate off of it either manually or via an automated tool that someone (likely the Babel team) would have to create.

I feared being responsible for people using a shiny, nascent syntax that Babel had made convenient without understanding the trade-offs. If users behaved as if this nascent syntax was set in stone, then the TC39 committee wouldn't feel as free to change it when necessary. But if no one tried out new syntax for fear of significant churn, then the committee wouldn't receive sufficient feedback. Ultimately, Babel had to handle this balance: I wrote a long post explaining the issue, forced users to opt in, provided a migration tool, and even renamed the packages "proposal" as a reminder. But that end result doesn't capture the struggle to get there: surrendering the need to please everyone and deciding on the necessary concessions.

In the two-year run-up to the release of Babel 7, there were days or weeks when all I felt was overwhelming anxiety: I was not doing enough, I was doing it wrong, I was repeating mistakes from the last release. I both wondered when it would be over and wished we could start fresh. Novelist Benjamin Nugent described a similar experience in a 2013 essay:

> When good writing was my only goal, I made the quality of my work the measure of my worth. For this reason, I wasn't able to read my own writing well. I couldn't tell whether something I had just written was good or bad, because I needed it to be good in order to feel sane. I lost the ability to cheerfully interrogate how much I liked what I had written, to see what was actually on the page rather than what I wanted to see or what I feared to see.

When my work became my ultimate goal, I suffered. My initial desire to help and share with others turned into a need to prove myself, and my passion for the work dwindled in the face of potential failure. I still struggle with putting all of my identity into what I do. Detachment felt impossible, especially as I moved from free time to full time.

When open source makes the news only after huge security breaches and even developers gloss over issues of project sustainability, the result is widespread ignorance about the complexity of maintenance. Users and new contributors often don't see, much less think about, the nontechnical issues—like mental health, or work-life balance, or project governance—that maintainers face. And without adequate support, our digital infrastructure, as well as the people who make it run, suffer.

Already there are too few maintainers in open source. So much of the conversation about increasing (and diversifying) our numbers focuses on reducing the barriers to entry. It's a valiant cause, but I still can't help but wonder: Are we just setting people up to fail?

When I first started out as an open-source contributor, I wanted to prove myself in an inspiring new community. I wasn't thinking of anyone's boundaries (their time, their responsibility to others, their emotions), least of all my own.

My first PR to JSCS in 2014—which later became the first project I helped maintain—was to add a table of contents. I tried out many approaches before submitting that PR: double- and triple- and quadruple-checking everything to make sure it looked perfect, copying examples of already merged changes, being as concise as possible. I looked up to the usernames at the top of the project's contributors list. Eventually, I got used to what I was doing—even got good at it. Getting to the top of the contributor list felt more possible than ever, yet still out of reach.

In May 2015, I got an email asking if I wanted to work at Adobe due to my work on JSCS. (By that time, I had also discovered Babel.) I decided to give it a shot: I figured that a company that found me through open source would encourage it on the job. Two of the core maintainers

of JSCS already worked at Adobe. To my delight, on my first day I was asked to join them.

When I became a maintainer of JSCS, I felt a new, shared sense of ownership—and the weight of a new burden. Suddenly, the people around me were no longer just names on a list but vitally important collaborators. This new awareness helped me realize that what I wanted was to help sustain not the code but the group itself. Now when I look at that list, it only reminds me of our bus factor. No longer a detached contributor, I cared less about my position on the leaderboard and more about the leaderboard itself as a holistic metric of the project's future.

A year after joining JSCS, I faced my first major decision as a maintainer. The JSCS team had to decide whether to stop the project or work with a similar one, ESLint. The difficulties were both technical and emotional. We had to settle on specifics regarding features, technical architecture, and vision. We also had to work through our feelings about something into which we had poured our time and our lives—our pride. After much discussion within the JSCS team and alongside ESLint, we were able to come to an agreement. We merged teams by deprecating the JSCS codebase and joining forces, much to the delight of the JavaScript community.

More complex challenges awaited me when I was invited to become a maintainer of Babel in November 2015. In many open-source organizations, maintainers play the part long before they're given the actual title. But when it is given, it helps to formalize the responsibility and effort. In the case of Babel, I didn't appoint myself or even ask to be named a maintainer; I was just offered collaborator rights. A couple months earlier, I had been given a shout-out in a much-anticipated 6.0 release, though at the time I didn't feel like I had done anything. (Only if I landed a commit to the project, enforced by GitHub's graphs and lack of visibility for non-coding work, did I count it as real work. I don't believe

this as much now.) Still, that simple encouragement inspired me to prove my worth in and to the Babel community. Being named a maintainer was the perfect way to begin.

When I was just a contributor, I could be happily ignorant of the project's overall vision. I knew someone else was there to handle it. As a maintainer, I began to carry an amorphous cognitive load that steadily increased. Each day the tasks piled up, the scope increased, the demands grew. As my responsibilities on Babel got bigger, I looked not for abstract tactics (such as filtering tasks) to learn but for exemplars to follow. I picked up on the habits of other maintainers who seemed able to follow every conversation and somehow also write blog posts, give talks, create videos, and start live streams. I remember Sebastian McKenzie, Babel's creator, was always able to answer the question or fix the bug at hand within five minutes. I attempted to do the same, albeit at a much slower pace. Eventually, other people's issues—first in the project itself, and then in everyone else's projects, via their mentions—became my issues. Every added medium of communication became another place to achieve inbox zero. And it really got to me: I felt like I couldn't be myself, couldn't have desires of my own.

I knew something was wrong, but, as a maintainer, it felt equally wrong to place obstacles between myself and others. Though I considered batching time in the morning versus checking throughout the day, turning off notifications, and not publishing anything before sleeping, creating barriers around myself felt like locking people out of the project, something that went against the "open" spirit of open source. But when I ignored my own boundaries, it came at a cost—to myself and eventually to those I wanted to help. My restless mind made it difficult to sleep and even caused physical signs like eczema flare-ups and backaches, painful but needed reminders of my own limitations.

If we continue as we are, who will maintain the maintainers? People may say that this burden is our responsibility, that we chose this job. They may also reduce our options down to one: If we don't like it, we should quit. But I think it's possible to honor my desire to serve others in a way that's sustainable.

What is the reasonable duty of a maintainer? Most of us don't even know *who* uses our project, whether they're corporations or individuals. We receive feedback or contributions from only a fraction of that user base. The lack of transparency is ironic, and the result is one of open source's great paradoxes: The ability to look at a program's source doesn't always incentivize others to support the code or the people behind it. Are we all not accountable?

Open source faces many challenges, from sources of funding (is it charity or business?) to questions of security (who is responsible, who pays the price?), and all demand our time and attention. But the question of maintainer mental health is one that is almost never discussed, even though it is just as important. Our digital infrastructure relies almost entirely on cycling through burned-out maintainers. As well as open-source projects may seem to run today, imagine what they might look like if their maintenance were performed by a stable, healthy workforce.

The tech industry ignores this problem at its own risk. For instance, Babel is maintained by six people, and only I work on it full time. Yet Babel is used by hundreds of companies and is even taught in colleges and bootcamps. It's not just a top-level dependency in applications: As a tool for writing JavaScript itself, it's used in both frameworks and libraries. Even if you don't use it yourself, your dependencies might: Babel has over 1.7 million dependent repos on GitHub. And Babel v7 has already surpassed v6, with over 8.5 million downloads per week on npm. Babel is

a key part of JavaScript's future, and it's just one of many essential building blocks powered by under-supported maintainers.

So how can maintainers offer more transparency about their workload and encourage more humane expectations from their user base? Too often, those with a need, request, or demand aren't aware of maintainers' competing priorities or past work on the same topic. Users in a queue should know that there is one, and where they are in it. GitHub's recently added status feature helps to signal busyness, but an improvement might look like a take-a-number system, a digital version of what you'd see in a deli. We should also make it easier for users to help themselves: to find answers that have already been addressed (and not just in an FAQ). Maintainers are not the only experts out there. We should empower *everyone* to share what they learn.

For maintainers, learning a skill like setting boundaries isn't as straightforward as memorizing data. In fact, it's more accurate to call this skill wisdom. Just learning something like programming (let alone open-source maintenance) can be overwhelming. Acquired through trial and (most of all) error, wisdom is even harder to gain. One place to begin is to ask: What are the rituals and practices—the liturgies—of our open-source community? To what end do they point us? How can we change them, or create them anew, to cultivate values like diligence, patience, and humility? To ensure the health of the people involved, and the long-term stability of the structures they build?

We are liturgical creatures: We work, shop, and travel according to cultural habit and holiday. It's not surprising that all religions follow a liturgical year to give time structure and, in some seasons, weight. In Christianity, there are the seasons of celebration (Easter and Christmas) and seasons of preparation (Lent and Advent). In many professions, work also shifts with the seasons. Farmers adapt to weather and time of year. Athletes give their all in competition and are aided by coaches and

trainers in their recovery. From education to tourism to retail, there are times of year when things slow down, when the load is lighter, or when the work requires a different sort of intensity. The equivalent for bodies might be fasting, for our digestion and metabolism; sleep, an opportunity to rest; or, for my spiritual community, the Sabbath, for spiritual contemplation and worship.

The digital realm, in contrast, moves at a constant pace, with little respite or opportunity to recharge. Maintainers know well the toll this takes, and many more makers and users of technology have experienced similar phenomena, even if they go unarticulated. Let's consider, then, a digital season. What would it look like to build space into our technological cycles? What could we accomplish if intentional rest was made ritual and regular?

Our community isn't so unique. The physical spaces (like cities) where people reside very much resemble our digital ones (like open source). We face the same issues of civility and hospitality. Whether a librarian, a public educator, or a local elected official, many of us have taken up roles where our primary goal is to better our community, to strengthen a shared resource, to contribute to the public good. The answers, for me, don't lie in code. Instead, I seek conversation across disciplines: to learn how others keep their projects and lives in balance, how they define their own boundaries, how they approach stewardship.

I should admit I'm still a work in progress. Ultimately, it's humility that helps me set boundaries, an awareness of where I fit into the grand story of life. If I can see myself as finite, limited in time, weak in certain areas, prone to mistakes, I will be better able to receive help from others, to delegate, and to set limits. It's precisely the boundaries from weakness, which comes from my admission of not knowing, that set me free. In open source, "the more work you do, the more work gets asked of you."

But just because work needs doing, doesn't mean it's your obligation or calling to do it (or to do all of it).

Open source is interrelated, dynamic, organic—not unlike us. And both are worth the pursuit to be made whole.

Learn Something New  Scaling & Growth  Ask an Expert

Interviews & Surveys  Guides & Best Practices

Essays & Opinion  Workplace & Culture

## ALL ISSUES

ISSUE 19 NOVEMBER 2021 — Planning

ISSUE 18 AUGUST 2021 — Mobile

ISSUE 17 MAY 2021 — Containers

ISSUE 16 FEBRUARY 2021 — Reliability

ISSUE 15 NOVEMBER 2020 — Remote

ISSUE 14 AUGUST 2020 — APIs

ISSUE 13 MAY 2020 — Frontend

ISSUE 12 FEBRUARY 2020 — Software Architecture

ISSUE 11 NOVEMBER 2019 — Teams

ISSUE 10 AUGUST 2019 — Testing

ISSUE 9 MAY 2019 — Open Source

ISSUE 8 FEBRUARY 2019 — Internationalization